Multi-Modal Chorus Recognition for Improving Song Search

Jiaan Wang[1], Zhixu Li[1*], Binbin Gu[4], Qingsheng Liu[5], Zhigang Chen[2,3]
[1]*Institute of Artificial Intelligence, School of Computer Science and Technology, Soochow University, Suzhou, China*
[2]*IFLYTEK Research, Suzhou, China*
[3]*State Key Laboratory of Cognitive Intelligence, iFLYTEK, Hefei, China*
[4]*University of California, Irvine, USA*
[5]*Anhui Toycloud Technology, Hefei, China*
Email: jawang1@stu.suda.edu.cn, zhixuli@suda.edu.cn, gu.binbin@hotmail.com, {qsliu, zgchen}@iflytek.com

***Abstract*—We discuss a novel task, Chorus Recognition, which could potentially benefit downstream tasks such as song search and music summarization. Different from the existing tasks such as music summarization or lyrics summarization relying on single-modal information, this paper models chorus recognition as a multi-modal one by utilizing both the lyrics and the tune information of songs. We propose a multi-modal Chorus Recognition model that considers diverse features and multi-modal fusion ways. Besides, we also create and publish the first Chorus Recognition dataset containing 627 songs for public use. Our empirical study performed on the dataset demonstrates that our approach outperforms several baselines in chorus recognition. In addition, our approach also helps to improve the accuracy of its downstream task - song search by more than 10.6%.**

I. Introduction

Nowadays, online music services have become mainstream ways for people to enjoy music. As a key function of online music services, song search aims to search for interesting songs by a segment of lyrics or tune. However, despite its importance, the song search capabilities offered by the existing applications are still unsatisfactory.

According to our study to several popular music applications [1]–[3], their song searches are often flawed in two ways. On the one hand, when the searching lyrics segment or tune segment is short and not representative enough, plenty of irrelevant songs might be returned by the song search. But on the other hand, when searching with a long lyrics or tune segment towards a large song library, the searching speed would be greatly slowed down. The reason for the above defects is that the existing song searches take fine-grained keywords or fragments of tunes as the basic searching unit. But most of the keywords or tune fragments are often shared by many songs such as the example shown in Figure 1, resulting in too many matching targets, thus reducing the efficiency and accuracy of song searches.

In this paper, we propose to improve the song search experience by identifying the most memorable or impressive part of a song, namely the chorus of the song. This allows the song search to focus primarily or preferentially on lyrics or tunes in the chorus part of songs. In this way, the length of the searchable part of songs could be greatly shortened, and the overlaps between the lyrics or tunes of different songs could be greatly reduced. As a result, both the accuracy and efficiency of the song search are expected to be enhanced.

Given the motivation above, we discuss a novel task called **Chorus Recognition**, aiming at identifying the chorus of a given song. We believe that this task could not only benefit song search but also other downstream tasks such as music summarization [4] that works on extracting the most representative parts of a music.

There are some other music-relevant tasks like music summarization [5] and lyrics summarization [6]. While music summarization just utilizes the tune of a music/song to identify the most representative parts, lyrics summarization only focuses on extracting or generating a short summary of the lyrics of songs. Either task only considers single-modal information. Thus, their approaches could not be directly adopted in chorus recognition. In order for better song search experience, we model Chorus Recognition as a multi-modal task where both lyrics and tune of songs would be taken into account.

Recent years have seen resounding success in multimodal learning [7], but all these supervised learning models usually require a large enough training data. Unfortunately, there is no publicly available dataset for the Chorus Recognition task. Some existing related datasets can not meet the needs of the Chorus Recognition task. For example, the RWC music dataset [8] which has been widely used in the music summarization task only has a manual annotation of the start and end times of each chorus section in each song. We can not obtain the lyrics information from the dataset.

In this work, we first build a CHOrus Recognition Dataset (CHORD), which contains 27k lyrics lines from 627 songs, each of which has been labeled with whether it belongs to the chorus part of a song. Then, based on this dataset, we propose the first multi-modal Chorus Recognition model which utilizes both the lyrics and tune information in different ways. Particularly, we denote a song with combined information in the following several ways. 1) Lyrics Embedding: Given the small size of the training data, we use the pre-trained language model (such as BERT [9]) and Graph Attention Network (GAT) [10] based on a constructed heterogeneous graph to get a better

* Zhixu Li is the corresponding author

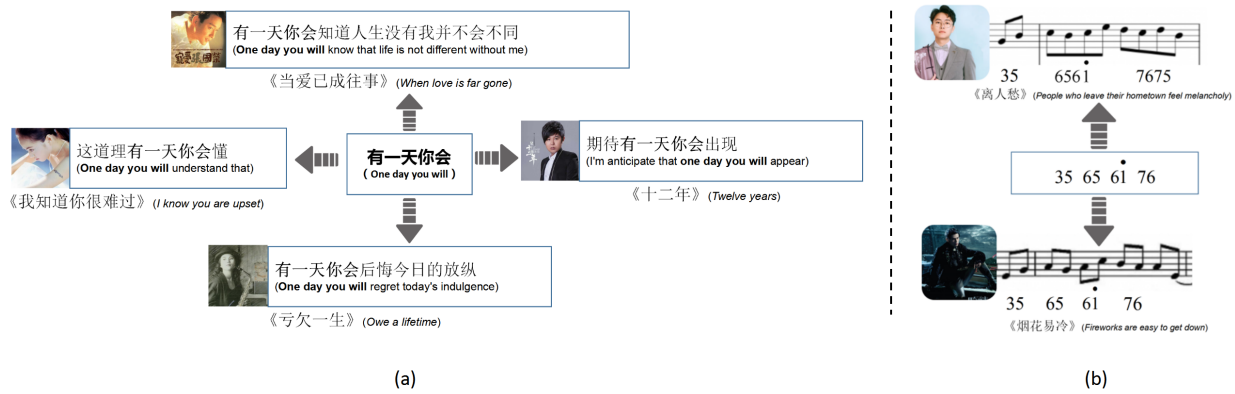

Fig. 1. (a) song search by lyrics keywords. (b) song search by tune fragments.

lyrics representation. 2) Tune Embedding: As for the tune of songs, the Mel Frequency Cepstrum Coefficient (MFCC) feature is adopted, which has been widely used in music summarization, speech recognition, and speaker recognition due to its robustness [11]. 3) Chord Embedding: Melody is a special feature for music that the other audio does not possess, and notes and chords are the basic components of a melody. Inspired by Word2vec [12], we train a skip-gram model on a large music dataset [13] and obtain pre-trained chord embedding. In this way, we can denote the melody of a music segment with a vector in a low-dimensional semantic space. Finally, two multi-modal fusion ways are discussed for combining the three embeddings of each lyrics line.

We summarize our contributions as follows:

- In order for better song search experience, we propose to do a novel upstream task called Chorus Recognition, aiming at identifying the chorus of a given song.
- We construct the first CHOrus Recognition Dataset (CHORD) and release it for further research[1].
- We propose a novel multi-modal Chorus Recognition model, where multi-modal features are employed, and different multi-modal fusion ways are discussed.

Our empirical study not only shows the effectiveness of our model in chorus recognition, but also demonstrates its effectiveness in improving the performance of song search with human evaluation.

The rest of this paper is orgnized as follows: we cover the related work in Section II, and then formally define the problem in Section III. Our approach is presented in Section IV, followed with dataset description in Section V. After presenting the experimental study in Section VI, we conclude the paper in Section VII.

II. Related Work

We first present the existing efforts on music/song search, and then cover two relevant tasks to chorus recognition including music summarization and lyrics summarization.

[1]The data and code has been made public, we will add the link immediately after the paper gets accepted.

A. Music/Song Search

The existing researches on music/song search either explore human music search behavior [14], [15], or focus on how to use the information to search for music [16]–[21].

Li et al. [14] reveal that users' behavior on music search is influenced by their search mindset. Hosey et al. [15] explore how and why people use search and how they evaluate their search experiences on a music-streaming platform.

Bahraini et al. [16] and Silva et al. [17] propose the method of searching sheet music respectively. Wang et al. [18] study how to use tags to query music. Buccoli et al. [19] explore how to search music through a text description. Leu et al. [20] and Chen et al. [21] make use of the audio segment to search for music.

As far as we are concerned, there is few influential research on the search mechanism for lyrics-based music/song search. The lyrics search function in the existing music apps basically borrows from the search mechanism of general search engine. However, different from the ordinary texts, lyrics are the carrier of melody. The fact that the lyrics lines and their corresponding melodies can be divided into intro, verse and chorus has never been explored and utilized by the existing work.

B. Music Summarization

Music summarization, also named as music thumbnailing, works on extracting a short segment of a piece of music that can represent the whole piece. Previous works typically assume that the melody pattern can represent the whole music/song if it appears most frequently. They use Self-Similarity Matrix (SSM) or Hidden Markov Model (HMM) methods to divide the song into several segments and then extracted he most frequent ones as the result [22], [23].

Nevertheless, many songs do not follow this assumption. To solve this problem, Bittner et al. [24] propose to do peak detection near the location where the user often pulls the progress bar to because users usually prefer the representative part of a song more. Also, Huang et al. [25] establish the

connections between emotions and summarization when well-defined and public domain labeled data for music summarization are unavaliable. Specifically, they use a music dataset with emotion labels to train a neural network model on music emotion classification task [26] and then add an attention mechanism on top of the networks to weigh the contribution of diverse short pieces of the song.

Mel Frequency Cepstrum Coefficient (MFCC) is widely applied in the music summarization [27]–[29]. In the first step, MFCC computes based on a perceptual-based frequency scale which refers to the human auditory model. Next, it outputs of a Mel-scale filter bank after obtaining the frame-based Fourier transform. Finally, they are decorrelated by means of the discrete cosine transform (DCT) [11]. For music, an audio often has a melody described by chords that the other audios do not possess. Although much work [30], [31] takes chord into account, they rarely pay attention to the music summarization. In our chorus recognition task, we both use the MFCC feature and chord embedding to represent a short piece of a song.

C. Lyrics Summarization and Extractive Text Summarization

Lyrics summarization aims to preserve key information and the overall meaning of a songtext [6]. As a special kind of extractive text summarization task, Fell et al. [6] propose to employ the generic extractive text summarization models over lyrics.

There are also other extractive text summarization models which extract important sentences from the original documents and reorganize them as the summary [32]. Traditional work uses the graph-based unsupervised ranking model, such as TextRank [33] and LexRank [34]. With the development of deep learning, Recently, many researchers turn to deep learning for help. Basically, these works are based on a two-stage framework. First, they use recurrent neural networks [35], [36] or Transformer encoders [37] to encode each sentence of the document. After that, they employ a classifier or hybrid networks to calculate the importance and redundancy of each sentence. In addition, some works leverage heterogeneous graph networks to deal with this task [32]. Recently, the emergence of pre-trained language model brings natural language processing to a new era [38]. Pre-trained model can learn knowledge from a large corpus on pre-defined tasks. Thereby, downstream tasks can benefit from these transferred knowledge, achieving better performance. Bidirectional Encoder Representations from Transformers (BERT) [9] is a well-known pre-trained language model which is based on transformer [39]. It learns a universal language encoder by pre-training with language modeling objective (i.e., predicting words that are masked out from the input based on the remaining context). Later, Liu et al. [40] added [CLS] token before each sentence in the document and input the whole document into BERT, so that each [CLS] token can aggregate the semantic information of the corresponding sentence through BERT. The representation of each [CLS] token (which represent each sentence) is then applied in text summarization task.

III. Problem Definition

The chorus of a song is usually the most representative part of the music/song, which usually consists of one or several segments of the music/song.

Given a music $M = \{(S_1, A_1), (S_2, A_2), \cdots, (S_k, A_k)\}$, where S_i denotes the i-th lyrics line, A_i denotes its corresponding audio in M and k represents the number of lyrics lines in M. The goal of Chorus Recognition is to decide whether (S_i, A_i) $(1 \leq i \leq k)$ belongs to the chorus part of M.

IV. Model

In this section, we present our model MMCR (**M**ulti-**M**odal **C**horus **R**ecognition) which consists of three parts. First, the skip-gram model [12] is pre-trained on the LMD-full dataset [13] for better chord embedding,. Secondly, we obtain the lyrics embedding based on Pre-trained Language Model and Graph Attention Networks [10]. Lastly, upon getting the final feature F_i for each (S_i, A_i), we use a classifier to calculate the possibility of each (S_i, A_i) belonging to the chorus part.

A. Audio Representation

We use both tune embedding and chord embedding to represent each audio A_i. Here we adopt the Mel Frequency Cepstrum Coefficient (MFCC) feature to represent the tune of songs. Note that although each type of audio has an MFCC feature, only music audio has chord feature which represents the melody of a music, so we use chord embedding to represent the melody of songs.

- **Tune Embedding**: The Mel Frequency Cepstrum Coefficient (MFCC) has been widely used in the field of speech recognition and speaker recognition [11]. We use MFCC feature as a basic feature for each audio A_i. The feature is obtained by an off-the-shelf python script. We denote the MFCC feature of each audio A_i as M_i.
- **Chord Embedding**: Inspired by skip-gram [12], we propose to train chord embedding with chord modeling task (given central chord to predict surrounding chord) on the LMD MIDI dataset [13] which contains about 170k midi files.
  Specifically, we first use an off-the-shelf script to extract the chord sequence of A_i, then we use pre-trained chord embedding to represent each chord in the chord sequence and concatenate each chord embedding to get a new chord feature.
  Formally, given the chord sequence $\{c_1, c_2, \cdots, c_{l_i}\}$, we compute the chord embedding CE_j for each c_j based on pre-trained chord embedding, where l_i represents the number of chords in A_i and c_j is a chord.

$$C_i = C(A_i) = CE_1 \oplus CE_2 \oplus \cdots \oplus CE_{l_i} \qquad (1)$$

where $\oplus$ means the concatenation operation.

B. Lyrics Embedding

In order to get lyrics embedding, we first use BERT [9] to obtain a basic semantic information of each lyrics line. After that, we construct a heterogeneous graph for each song and use Graph Attention Network (GAT) [10] to enrich global information for each lyrics line. Finally, the sinusoid position embedding is added to represent the position of the lyrics line in the whole music/song.

More specifically, given a lyrics line S_i, we input the whole sequence to BERT which then automatically insert [CLS] token to the beginning of S_i and calculate the representation of each token through stacked transformer encoders [10], Then, we use the output representation of token [CLS] to represent the basic semantic information of S_i because this representation aggregates information from the whole lyrics line through stacked transformer encoders. We use L_{Bert_i} to represent lyrics embedding of each lyrics line S_i through BERT [9].

Note that, L_{Bert_i} only contains the semantic information of each lyrics line S_i itself, which means L_{Bert_i} does not contain other lyrics lines information from the same song. Intuitively, different lyrics lines in one song can be complementary to each other. For example, if two lyrics lines have similar words or tune, we can use one lyrics line to enrich the representation of another lyrics line , making the lyrics embedding more robust. This suggests that we can enrich the representation of each lyrics line by global information (i.e. the other lyrics lines in the same song). Inspired by HSG [32], we use Graph Attention Networks [10] to show how we achieve this purpose.

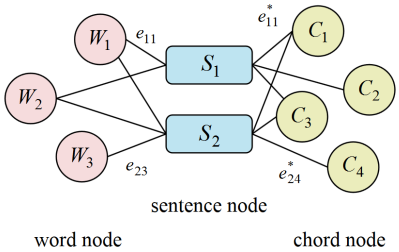

Fig. 2. The heterogeneous graph for a song.

As shown in Figure 2, given music/song M, we first construct a heterogeneous graph which consists of three types of nodes: sentence nodes, word nodes, and chord nodes. Each sentence node corresponds to a lyrics line in M. For each word (or chord) in the lyrics (or tune) of M, we create a word (or chord) node for it. Each sentence node connects with the word nodes if the sentence contains the words. In a similar way, each sentence node also connects with the chord nodes if the chord is contained in the tune behind the sentence. The graph takes the importance of relationships as their edge feature. We denote e_{ij} as the edge feature between word nodes and sentence nodes and e^*_{ij} as the edge feature between chord nodes and sentence nodes.

We denote L_{Bert_i} as the initial value for each sentence node and initialize each word node with corresponding GloVe embeddings [41]. For chord nodes, we set the chord embedding as the initial value which is discussed in section IV-A. Besides, we input TF-IDF values as the edge weights which indicate the importance between each pair of nodes.

We use graph attention networks [10] to update the representations of each node in the heterogeneous graph. Formally, given a constructed graph G with node features $X_w \cup X_s \cup X_c$ and edge features E, the graph attention (GAT) layer is designed as follows:

$$Z_{ij} = LeakyReLU(W_a[W_q h_i; W_k h_j; e_{ij}]) \tag{2}$$

$$\alpha_{ij} = \frac{exp(z_{ij})}{\sum_{l \in N_i} exp(z_{il})} \tag{3}$$

$$u_i = \sum_{j \in N_i} \alpha^k_{ij} W^k h_j \tag{4}$$

h_i is the hidden states of input nodes, W_a,W_q,W_k,W_v are trainable weighs. α_{ij} means the attention weight between h_i and h_j. We calculate it based on h_i, h_j and the corresponding edge feature e_{ij}. After that, we use the attention weight to aggregate other nodes' information (u_i) for the central node. In addition, the multi-head attention is applied which can be formulated as follow:

$$u_i = \|_{k=1}^{K} \, (\sum_{j \in N_i} \alpha^k_{ij} W^k h_j) \tag{5}$$

where $\|_{k=1}^{K}$ means multi-head attention. After obtaining the additional information (u_i) for the central node, we update the representation of central node by combining the original representation h_i and the additional information u_i as follows:

$$h_i^{'} = u_i + h_i \tag{6}$$

where $h_i^{'}$ is the updated representation of central nodes. Inspired by transformer [39], we leverage a position-wise feed-forward (FFN) layer which consists of two linear transformations after each graph attention layer mentioned above.

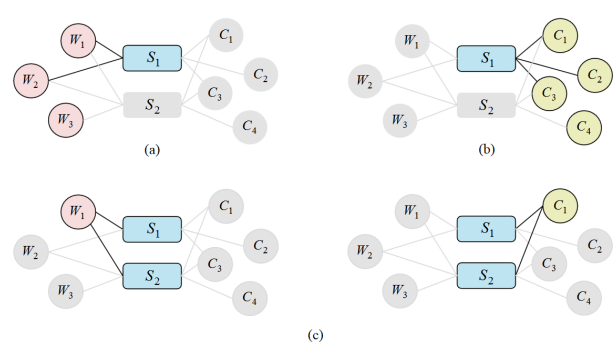

Fig. 3. The information propagation order in the heterogeneous graph.

As shown in Figure 3, the information propagation order is built in the heterogeneous graph. Firstly, we use word nodes to enrich the representation of sentence nodes. Secondly, the sentence nodes are enriched by chord nodes. Lastly, we enrich the representation of word nodes and chord nodes with

the representation of sentence nodes. After completing the heterogeneous graph, each lyrics line obtains the global semantic information from the whole music/song M. We denote the contextual representation for each lyrics line as L_{HG_i}. For each lyrics line, we add sinusoid positional embeddings (function $PosEmb$) [39] to represent the position of the lyrics line in the whole music/song M. We denote L_i the final lyrics embedding which is calculated in the following equation.

$$L_i = PosEmb(L_{HG_i}) \tag{7}$$

C. Multi-modal Fusion for Classification

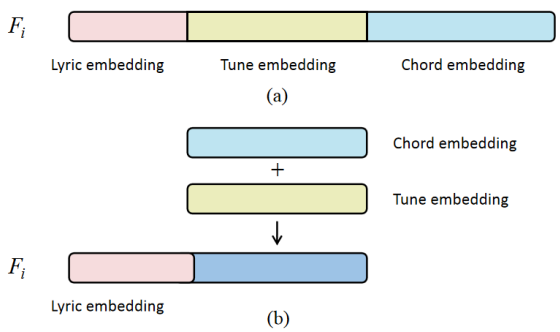

Fig. 4. (a) the first multi-modal fusion way, (b) the second fusion way.

While three features (lyrics embedding, tune embedding and chord embedding) have been obtained seperately, we now introduce how to exploit the three features. As shown in Figure 4, we explore the following ways to integrate these three features:

- In the first way, we concatenate lyrics embedding L_i, tune embedding M_i and chord embedding C_i. The final representation of (S_i, A_i) then can be formulated as follows:
$$F_i = L_i \oplus M_i \oplus C_i \tag{8}$$
- In a second way, we concatenate lyrics embedding L_i and audio representation to obtain the final representation of (S_i, A_i), while audio representation is obtained by combining the tune embedding M_i and the chord embedding C_i with different weights. Formally:
$$F_i = L_i \oplus (\lambda_1 M_i + \lambda_2 C_i) \tag{9}$$
$$\lambda_1 + \lambda_2 = 1 \tag{10}$$

After obtaining the final representation of each (S_i, A_i), we put the feature F_i to several fully connected layers with ReLU activation function, and gradually reduce the dimensions of the fused feature to one. We denote y_i as the possibility of (S_i, A_i) belonging to the chorus part of the given music/song M. We then use the sigmoid activation function to calculate y_i as follows.

$$y_i = sigmoid(W_2 ReLU(W_1 F_i + b_1) + b_2) \tag{11}$$

Finally, we update the weight of model-based cross-entropy loss between y_i and the ground truth value.

V. Dataset

In order to train and evaluate different models, we construct CHOrus Recognition Dataset (CHORD) which contains 27k lyrics sentence from 627 songs. This section demonstrates how we collect data, annotate data and guarantee the quality of the dataset.

A. Dataset Collection Process

We collected different types of popular Chinese song lists in QQ music[2] and downloaded all the songs in the popular song list. Due to the copyright of some music, we only reserved the songs available for free download. We randomly selected the final 1000 songs as the original data for building CHORD. These songs are representative ones for many genres of music such as rock, pop, classical, rap, jazz, and so on.

After a song is downloaded, two related files are available: MP3 file and LRC file. The MP3 file stores all the audio information of the song. In order to extract the audio features conveniently, we use the off-the-shelf tools to convert the MP3 audio file into a WAV audio file, the LRC file saves the lyrics and the corresponding time information.

B. Dataset Annotation Process

In order to annotate the data more conveniently and quickly, we built an internal data annotation platform and developed a strict annotation standard of the chorus part to guide workers to annotate the data. Many workers annotate the lyrics lines on the platform. Based on the corresponding audio files, each lyrics line is marked as "0" or "1". "0" represents the lyrics line is not in the chorus part, and "1" represents the opposite. Figure 5 shows an example of how the annotation is performed. At first, several workers are asked to annotate the sampled data at the same time. Next, we evaluate the workers' performance based on their answers of all the samples. Only those workers with high performance (i.e. high accracy) can continue to annotate the rest of the data[3].

During the process of annotating data, each song will be assigned to three different workers to label separately. If two or more people are consistent, the label of the song will be passed directly. Most of the annotation results will be taken as the final result. If the annotation results of the three persons are not the same, the final annotation will be confirmed by the data specialists.

In the end, each song will be checked by data specialists. As long as one specialist thinks it does not meet the requirements, it will be re-annotated until all specialists think that it completely meets the requirements.

C. Data Statistics

After the data annotation process is finished, we only keep 627 songs because some song files are invalid or some of them are too short (e.g. less than 60 seconds). Finally, CHORD

[2]https://y.qq.com/

[3]We have 22 out of 25 annotators pass our music annotation qualification test. All these annotators are undergraduate students in the music school of our university.

Lyrics Line	Annotation
在千里以外在呼喊的是什么 (What is calling out thousands of miles away)	0
在百年以后想回忆的是什么 (What do you want to remember in a hundred years)	0
在离开以前能否再见那一刻 (Can I see that moment again before I leave)	0
记得你的眼睛将会亮着 (Remember your eyes will be on)	1
我的手臂将会挥着 (My arm will swing)	1
谁说世界早已没有选择 (Who says the world has no choice)	1
趁着我会喜怒你会哀乐 (While we are both sad and happy)	1
唱几分钟情歌 (Sing love songs for a few minutes)	1
没什么至少证明我们还活着 (Nothing at least proves we're alive)	1
像单纯的蝴蝶为玫瑰的甜美而飞着 (Like pure butterflies flying for the sweetness of roses)	0

Fig. 5. the part of lyrics of *model* with corresponding annotation.

contains 27k lyrics lines and each song contains an average of 43.17 lyrics lines. In our experiments, we divide CHORD into train, validation and testing sets with a rough ratio of 80/10/10.

D. Data Quality

We conduct human evaluations for the quality of the data. If the annotation of a song is consistent with the established annotation standard, and the annotation conforms to common sense knowledge about music, it will get 1 point, otherwise it will get 0 point. We randomly selected 200 songs from CHORD and asked five people to grade them. Finally, we obtain an average score of 0.94.

VI. Experiments

A. Implementation Detail

We set our model parameters based on the preliminary experiments on the validation set. For extracting lyrics embedding, we use the improved version of BERT: BERT-wwm-ext [42] which sets the dimension of word embeddings to 512, feed-forward units to 2048, 12 heads for multi-head self-attention and the number of transformer encoder layer to 12. To build the heterogeneous graph, we limit the size of vocabulary to 50000 and only 12 most common chords are used in chord nodes. We initialize word nodes with 300-dimensional GloVe embeddings [41]. We remove 10% of the vocabulary with low TF-IDF values over the whole dataset to get rid of the noisy common words. This strategy is also used in HSG [32]. We pre-train the parameters of the graph attention (GAT) layer on our music database (which contains about 370k popular Chinese songs) with the self-supervised task: *next lyrics line prediction*. Similar to the next sentence prediction (NSP) [9], given two lyrics lines A and B, the next lyrics line prediction is used to judge whether B is the next lyrics line of A. The representation of each lyrics line is obtained by the graph attention layer. In the chorus recognition task, we fix the parameters of the graph attention layer for fair comparison.

For MFCC feature of each audio A_i, we use the python script[4] to get MFCC feature $M_i \in R^{t_i \times 13}$, t_i (the first dimension of MFCC feature) is decided by the length of A_i. Specifically, we prune t_i to 1280. If t_i is less than 1280, we apply 0 pad to set this dimension to 1280. In addition, we map the second dimension of MFCC feature into 1 with two fully connected layers.

For pretrained chord embedding, we first limit the size of chord vocabulary to 500 in the skip-gram model [12] and each chord has a 64-dimensional pretrained embedding. Next, we leverage the off-the-shelf script[5] to extract chord sequences from a part (around 12k midi files) of the LMD-full dataset [13]. Finally, we train skip-gram model on the chord sequences. For chord embedding of each A_i, we utilize the off-the-shelf script[6] to extract chord sequence of A_i and only retain the first 20 chords as the number of chords in 95% of the audio A_i is less than 20.

In MMCR, we use Adam optimizer [43] with an initial learning rate $\alpha = 0.0006$, momentum $\beta_1 = 0.9$ and $\beta_2 = 0.999$. And the minimal batch size is set as 128.

B. Metrics and Approaches

We compare our model with several typical baselines and several baselines proposed in the latest years in terms of accuracy, precision, recall and F1 score.

1) **TextRank** [33]: TextRank is an unsupervised graph-based method that computes each sentence's importance score based on eigenvector centrality within weighted-graphs for extractive text summarization.

2) **PacSum** [44]: PacSum is also an unsupervised text summarization algorithm. Different from TextRank, PacSum builds graphs with directed edges and employs BERT to better capture sentential meaning.

3) **Ext-BERT**: Extractive summarizer with BERT learns the semantic of each lyrics line in a purely data-driven way. The method calculates each lyrics line importance score based on their semantics.

4) **RNAM-LF** [25]: Recurrent Neural Attention Modeling by Late Fusion is a supervised music thumbnailing algorithm proposed in recent years. RNAM-LF allows us to get the music highlight span, providing the most likely range of the chorus part.

C. Results

As shown in Table I, MMCR significantly improves the performance compared with other approaches. The first part of Table I (the horizontal lines divide the table into four parts) contains two unsupervised text summarization methods. These two methods can only calculate the importances scores of lyrics lines and do not know how many lyrics lines should be extracted from each song, so we only keep the most important K lyrics lines as a result. K represents how many lyrics lines belong to the chorus part in each song which from

[4]https://github.com/jameslyons/python_speech_features
[5]https://github.com/alcolado/chord2vec
[6]https://github.com/yashkhem1/Chord-Extraction-using-Machine-Learning

the testing set of CHORD (different song has different K value). The second part is the supervised models based on lyrics embedding. Ext-BERT has been explained above, Ext-BERT-wwm-ext is just replace BERT [9] with BERT-wwm-ext [42]. RNAM-LF [25] is a music thumbnailing algorithm. Since it has been trained on tens of thousands of music data on tasks different from ours, we used the pre-trained RNAM-LF directly to get the highlighted span in music. After obtaining the highlight span of music, only if half of a lyrics line is within this span, we then predict that the lyrics line belongs to the chorus part.

TABLE I
PERFORMANCE COMPARISON OF DIFFERENT MODELS ON CHORD.

Model	ACC	P	R	F_1
TextRank	49.21	48.15	48.15	48.15
PacSum	64.09	63.34	63.34	63.34
Ext-BERT	69.65	68.17	71.35	69.73
Ext-BERT-wwm-ext	70.39	69.16	71.35	70.24
RNAM-LF	67.87	67.21	67.17	67.19
MMCR(BERT)	85.44	**86.56**	83.19	84.84
MMCR(BERT-wwm-ext)	**85.94**	85.52	**85.83**	**85.67**

D. Ablation Study

To evaluate the effectiveness of each feature (lyrics embedding, tune embedding or chord embedding), we removed some features respectively from our model on the testing set. Specifically, we use lyrics embedding, tune embedding (MFCC feature), chord embedding, lyrics embedding and tune embedding (MFCC feature). To prove BERT-wwm-ext is better than BERT, we also replace BERT-wwm-ext with BERT in our model.

TABLE II
RESULT OF ABLATION EXPERIMENT ON CHORD.

Model	ACC	P	R	F_1
Chord (Random)	54.25	53.03	57.60	55.22
Chord (Skip-gram, fix)	55.89	54.71	57.66	56.14
Chord (Skip-gram, fine-tune)	56.06	54.36	64.11	58.84
Ext-BERT	69.65	68.17	71.35	69.73
Ext-BERT-wwm-ext	70.39	69.16	71.35	70.24
MFCC	81.84	80.54	84.38	82.42
MFCC+Lyrics (BERT)	85.21	84.71	85.17	84.94
MFCC+Lyrics (BERT-wwm-ext)	85.56	84.86	**85.83**	85.34
MMCR (BERT)	85.44	**86.56**	83.19	84.84
MMCR (BERT-wwm-ext)	**85.94**	85.52	**85.83**	**85.67**

The result of the Ablation Study as shown in Table II, each model is explained as follows:

1) **Chord (Random)**: The model only uses chord embedding. Specifically, we randomly initialize the chord embedding and update the value during the process of training.

2) **Chord (skip-gram, fix)**: The model only uses chord embedding. Specifically, we train skip-gram model to obtain pre-trained chord embedding and use trained embedding as a fixed value when we train the model.

3) **Chord (skip-gram, fine-tune)**: The model only uses chord embedding. Specifically, we update the pre-trained embedding values in our task.

4) **Ext-BERT** and **Ext-BERT-wwm-ext**: The same as the models we introduce above. These two models only use lyrics embedding.

5) **MFCC**: The model only considers the MFCC feature to solve the chorus recognition task.

6) **MFCC+Lyrics**: This approach combines MFCC feature and lyrics embedding, the lyrics embedding has been extracted from BERT [9] or BERT-wwm-ext [42] respectively.

As can be seen from Table II, chord embedding plays a limited role in the chorus recognition task. There may be errors when using an off-the-shelf script to extract chords, because the script uses a traditional machine learning algorithm to predict chord for each 0.5s audio. However, chord embedding can bring greater improvement if we can guarantee the accuracy of chord extraction. MFCC is the most important feature among these three features (lyrics embedding, tune embedding and chord embedding), as we find that MFCC achieves much better performance when only one of three features is used. The results also demonstrate that we can use lyrics embedding and chord embedding to make the overall representation more robust and achieve better performance. Finally, the results indicate that BERT-wwm-ext is better than BERT in our task.

E. Multi-modal Fusion Study

To achieve better performance, we also explored different multi-modal fusion ways. As we mentioned in Section IV-C, we use Lyrics embedding L_i, Tune embedding (MFCC feature) M_i and Chord embedding C_i to calculate the final feature F_i for each (S_i, A_i). The results of the multi-modal fusion study is shown in Table III. MF_1 means the first multi-modal fusion approach and MF_2 means the second. Although the results of these two methods are similar, we find that the second method has less parameter than the first one because the parameters of fully connected layers in the second way is less than the first one. These two approaches are relatively simple which make the difference of results small. We use them in our model because of the limited amount of data.

TABLE III
RESULT OF MULTIMODAL FUSION STUDY ON CHORD.

	Model	ACC	P	R	F_1
MF_1	MMCR (BERT)	85.44	86.56	83.19	84.84
	MMCR (BERT-wwm-ext)	**85.94**	85.52	**85.83**	**85.67**
MF_2	$\lambda_1 = 1.00$ $\lambda_2 = 0.00$	85.56	84.86	**85.83**	85.34
	$\lambda_1 = 0.50$ $\lambda_2 = 0.50$	85.71	86.05	84.51	85.27
	$\lambda_1 = 0.70$ $\lambda_2 = 0.30$	85.91	86.56	84.33	85.43
	$\lambda_1 = 0.90$ $\lambda_2 = 0.10$	85.82	86.99	83.55	85.23
	$\lambda_1 = 0.95$ $\lambda_2 = 0.05$	85.62	**87.35**	82.60	84.91

F. Evaluation on Song Search

We also demonstrate the effectiveness of our model in the song search task with human evaluation. There are many

pieces of informations that can be used to search a song, such as a song title, lyrics, tunes and so on. Specifically, our song search experiment uses keywords to search for songs.

TABLE IV
RESULT OF SONG SEARCH TASK.

Applications/Methods	Top-1	Top-3
Youtube Music	0.73	0.84
QQ Music	0.75	0.86
Netease Cloud Music	0.69	0.79
TF-IDF	0.55	0.71
MMCR(our)	**0.83**	**0.91**

We compare our model with several applications and typical baselines:

1) **MMCR**: We calculate the chorus probability of each lyrics line by MMCR on our music database which has about 370k popular Chinese songs. Several lyrics lines from different songs may contain the same keywords input. We return the song whose lyrics line has the maximum chorus probability.

2) **TF-IDF**: The term frequency (TF) is the number of times w_i occurs in S_j and the inverse document frequency (IDF) is made as the inverse function of the out-degree of w_i [32]. When several lyrics lines from different songs contain the keywords, we compute the average keywords TF-IDF value for each song and return the song with the highest value.

3) **Youtube Music**[7], **QQ Music**[8] and **Netease Cloud Music**[9]: All of the three music applications provide song search services for the users. Among them, Youtube Music is extremely popular and serves the worldwide users. QQ Music and Netease Cloud Music are the two most popular music applications in China.

In our manual evaluation, we construct candidate keywords set for each song in our music database. Given a song, we first extract chorus lyrics lines by MMCR. Then we collect all three or four consecutive words from each chorus lyrics line. If the consecutive words appear in the non-chorus part of at least two other songs, it will be added to the candidate keywords set of the given song. We choose three or four consecutive words as keywords for the following reasons: (1) too few keywords may cause Music APP to identify a song as its title; (2) too many number of words in the keywords leads to the probability of keywords appearing in other songs dropping significantly.

The specific process of human evaluations is as follows:

1) Choose a song from our music database which has about 370k popular Chinese songs.
2) Choose keywords from the candidate keywords set of the song selected in the first step.
3) Use the keywords to search songs by all applications and methods.
4) We give one point to each application or method if the corresponding search result is the same as the song selected in the first step and zero point for otherwise.

[7]https://music.youtube.com/
[8]https://y.qq.com/
[9]https://music.163.com/

We also make the following restrictions to ensure the fair comparison:

- Song search in the applications (Youtube Music, QQ Music and Netease Cloud Music) needs to be judged as lyrics search. Because the number of words in keywords is small, sometimes the keywords input are identified as a song title.
- The search result of applications only keep songs which are also in our music database.
- All the applications are not allowed the users to log in because the search results may depend on the users' preference due to the built-in recommendation system.
- Each song can be selected at most once.

We ask 30 volunteers to do manual evaluation for 20 times per person. The result of the song search task is shown in Table IV. Each score represents the average result of human evaluation. Top-1 means only the correct song showing up at the first place will get 1 point, and Top-3 represents methods or applications can get 1 point if the correct song is in the top three of the result list. As we can see, our method achieves better performance in the scenario of song search by keywords (extract from lyrics). Specifically, TF-IDF only use the sinlge-modal information (lyrics), so the corresponding result is not competitive. We notice that the lyrics lines and their corresponding melodies can be divided into intro, verse and chorus. However, these important information has never been explored and utilized by existing work. So, our approach achieves better performance than these methods. Specifically, our model improves the accuracy of this task by more than 10.6% compared with the second best approaches.

VII. Conclusion and Future Work

In this work, we develop a multi-modal chorus recognition model based on multi-modal fusion. Through the improved BERT and graph attention networks, we achieved better lyrics embedding. Also, by leveraging the pre-trained chord embedding we enhanced the performance of the model and explored different ways of feature fusion. Experimental results show that our model achieves better performance compared to existing approaches.

In the future we will study the following work:

- How to use chord feature more effectively and develop more effective multi-modal fusion approaches.
- Evaluate the potential usefulness of the title of a song.

References

[1] M. Eriksson, R. Fleischer, A. Johansson, P. Snickars, and P. Vonderau, *Spotify teardown: Inside the black box of streaming music*. Mit Press, 2019.

[2] D. Hesmondhalgh, E. Jones, and A. Rauh, "Soundcloud and bandcamp as alternative music platforms," *Social Media + Society*, vol. 5, 2019.

[3] X. Amatriain, "Clam: A framework for audio and music application development," *IEEE Software*, vol. 24, 2007.

[4] X. Shao, C. Xu, Y. Wang, and M. Kankanhalli, "Automatic music summarization in compressed domain," *2004 IEEE International Conference on Acoustics, Speech, and Signal Processing*, vol. 4, pp. iv–iv, 2004.

[5] D. Hu, F. Nie, and X. Li, "Dense multimodal fusion for hierarchically joint representation," *2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 3941–3945, 2019.

[6] M. Fell, E. Cabrio, F. L. Gandon, and A. Giboin, "Song lyrics summarization inspired by audio thumbnailing," in *International Conference Recent Advances in Natural Language Processing*, 2019.

[7] C. Zhang, Z. Yang, X. He, and L. Deng, "Multimodal intelligence: Representation learning, information fusion, and applications," *IEEE Journal of Selected Topics in Signal Processing*, 2020.

[8] M. Goto, H. Hashiguchi, T. Nishimura, and R. Oka, "Rwc music database: Popular, classical and jazz music databases," in *International Society for Music Information Retrieval*, 2002.

[9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186.

[10] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," *arXiv preprint arXiv:1710.10903*, 2018.

[11] F. Alías, J. C. Socoró, and X. Sevillano, "A review of physical and perceptual feature extraction techniques for speech, music and environmental sounds," *Applied Sciences*, vol. 6, p. 143, 2016.

[12] T. Mikolov, K. Chen, G. S. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.

[13] C. Raffel, "Learning-based methods for comparing sequences, with applications to audio-to-midi alignment and matching," Ph.D. dissertation, Columbia University, 2016.

[14] A. Li, J. Thom-Santelli, P. Chandar, C. Hosey, B. Thomas, and J. Garcia-Gathright, "Search mindsets: Understanding focused and non-focused information seeking in music search," *The World Wide Web Conference*, 2019.

[15] C. Hosey, L. Vujovic, B. Thomas, J. Garcia-Gathright, and J. Thom-Santelli, "Just give me what i want: How people use and evaluate music search," *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, 2019.

[16] A. Bahraini and E. Tilevich, "Ask toscanini!: architecting a search engine for music scores beyond metadata," *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*, 2019.

[17] D. D. Silva and S. Hettiarachchi, "Analyzing the perception of online sheet music search: A case study," 2018.

[18] J.-C. Wang, Y.-C. Shih, M.-S. Wu, H.-M. Wang, and S.-K. Jeng, "Colorizing tags in tag cloud: a novel query-by-tag music search system," in *Proceedings of the 19th ACM international conference on Multimedia*, 2011, pp. 293–302.

[19] M. Buccoli, M. Zanoni, A. Sarti, and S. Tubaro, "A music search engine based on semantic text-based query," *2013 IEEE 15th International Workshop on Multimedia Signal Processing (MMSP)*, pp. 254–259, 2013.

[20] J. Leu, C. Changfan, K.-W. Su, and C. Chen, "Design and implementation of a fixed-mobile convergent music search engine (fmc-muse)," *Wireless Personal Communications*, vol. 70, pp. 1911–1923, 2013.

[21] X. Chen and X. Kong, "An efficient music search method in large audio database," *2018 3rd International Conference on Mechanical, Control and Computer Engineering (ICMCCE)*, pp. 484–487, 2018.

[22] M. Bartsch and G. Wakefield, "Audio thumbnailing of popular music using chroma-based representations," *IEEE Transactions on Multimedia*, vol. 7, pp. 96–104, 2005.

[23] M. Cooper and J. Foote, "Summarizing popular music via structural similarity analysis," *2003 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (IEEE Cat. No.03TH8684)*, pp. 127–130, 2003.

[24] R. M. Bittner, M. Gu, G. Hernandez, E. J. Humphrey, T. Jehan, H. McCurry, and N. Montecchio, "Automatic playlist sequencing and transitions," in *International Society for Music Information Retrieval*, 2017.

[25] Y. Huang, S.-Y. Chou, and Y. Yang, "Pop music highlighter: Marking the emotion keypoints," *Trans. Int. Soc. Music. Inf. Retr.*, vol. 1, pp. 68–78, 2018.

[26] Y. Yang and J. Liu, "Quantitative study of music listening behavior in a social and affective context," *IEEE Transactions on Multimedia*, vol. 15, pp. 1304–1315, 2013.

[27] K. A. Meintanis and F. M. Shipman, "Creating and evaluating multi-phrase music summaries," in *International Society for Music Information Retrieval*, 2008.

[28] M. Guo and H. Zhang, "Automatic summarization for popular songs," *2008 International Conference on Audio, Language and Image Processing*, pp. 653–658, 2008.

[29] S. Gao and H. Li, "Popular song summarization using chorus section detection from audio signal," in *2015 IEEE 17th International Workshop on Multimedia Signal Processing (MMSP)*. IEEE, 2015, pp. 1–6.

[30] H. Zhu, Q. Liu, N. J. Yuan, C. Qin, J. Li, K. Zhang, G. Zhou, F. Wei, Y. Xu, and E. Chen, "Xiaoice band: A melody and arrangement generation framework for pop music," *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2018.

[31] F. Pachet, A. Papadopoulos, and P. Roy, "Sampling variations of sequences for structured music generation," in *International Society for Music Information Retrieval*, 2017.

[32] D. Wang, P. Liu, Y. Zheng, X. Qiu, and X. Huang, "Heterogeneous graph neural networks for extractive document summarization," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Online: Association for Computational Linguistics, Jul. 2020, pp. 6209–6219.

[33] R. Mihalcea and P. Tarau, "TextRank: Bringing order into text," in *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*. Barcelona, Spain: Association for Computational Linguistics, Jul. 2004, pp. 404–411.

[34] G. Erkan and D. R. Radev, "Lexrank: Graph-based lexical centrality as salience in text summarization," *arXiv preprint arXiv:1109.2128*, 2004.

[35] R. Nallapati, F. Zhai, and B. Zhou, "Summarunner: A recurrent neural network based sequence model for extractive summarization of documents," 2017.

[36] Q. Zhou, N. Yang, F. Wei, S. Huang, M. Zhou, and T. Zhao, "Neural document summarization by jointly learning to score and select sentences," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Melbourne, Australia: Association for Computational Linguistics, Jul. 2018, pp. 654–663.

[37] M. Zhong, D. Wang, P. Liu, X. Qiu, and X. Huang, "A closer look at data bias in neural extractive summarization models," in *Proceedings of the 2nd Workshop on New Frontiers in Summarization*. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 80–89.

[38] X. Qiu, T. Sun, Y. Xu, Y. Shao, N. Dai, and X. Huang, "Pre-trained models for natural language processing: A survey," *arXiv preprint arXiv:2003.08271*, 2020.

[39] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in neural information processing systems*, 2017, pp. 5998–6008.

[40] Y. Liu and M. Lapata, "Text summarization with pretrained encoders," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 3730–3740.

[41] J. Pennington, R. Socher, and C. Manning, "GloVe: Global vectors for word representation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Doha, Qatar: Association for Computational Linguistics, Oct. 2014, pp. 1532–1543.

[42] Y. Cui, W. Che, T. Liu, B. Qin, Z. Yang, S. Wang, and G. Hu, "Pre-training with whole word masking for chinese bert," *arXiv preprint arXiv:1906.08101*, 2019.

[43] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[44] H. Zheng and M. Lapata, "Sentence centrality revisited for unsupervised summarization," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Florence, Italy: Association for Computational Linguistics, Jul. 2019, pp. 6236–6247.